



Midterm Project Proposal

Fine-Tuning LLM Models for Bug Detection in Quantum Software

CSC 7644

Submitted By: Md Saidur Rahman

Email: mrahm65@lsu.edu

Date: April 12, 2026

Contents

1	Introduction & Motivation	2
1.1	Project Overview	2
1.2	Project Deliverables	2
2	Background	2
2.1	Quantum Bug Benchmarks	2
2.2	LLM-Based Bug Detection	2
2.3	Research Gap	3
3	Proposed Technical Approach	3
3.1	Method Justification	3
3.2	Pipeline	3
3.3	Tools and Techniques	3
4	Data & Resources	3
5	Evaluation Plan	4
6	Feasibility & Risks	4
7	Ethical, Privacy & Security Considerations	4
	References	5

Chapter 1

Introduction & Motivation

1.1 Project Overview

Quantum computing is first transitioning from theoretical research to practical software development, with frameworks such as IBM’s Qiskit enabling developers to write and execute quantum programs. However, quantum programs are susceptible to domain-specific bugs—incorrect qubit mappings, gate misuse, measurement ordering errors, and entanglement violations that fundamentally differ from classical software bugs. These quantum-specific faults are difficult to detect using traditional debugging tools, which lack awareness of quantum semantics (Zhao et al., 2021).

An existing open-source research framework, *quantum-bug-rag*, provides prompt-only and retrieval-augmented generation (RAG) baselines for LLM-based quantum bug detection. However, neither mode internalizes domain-specific bug patterns into the model’s weights. Fine-tuning offers a promising alternative: by training directly on labeled quantum bug data, the model can learn structural patterns unique to Qiskit programs, potentially outperforming both prompt-only and retrieval-based approaches.

This project proposes extending the existing framework with a fine-tuning mode using parameter-efficient methods (LoRA/QLoRA) on open-source LLMs. The primary audience includes quantum software developers, software engineering researchers, and educators in quantum computing courses. From a security perspective, undetected quantum bugs in applications such as quantum key distribution (QKD) can cause silent failures that compromise cryptographic guarantees, making automated detection tools critical.

1.2 Project Deliverables

- **Fine-tuned LLM models** for quantum bug detection and classification:
 - LoRA/QLoRA fine-tuned Llama 3.1 8B on Bugs4Q dataset
 - LoRA/QLoRA fine-tuned DeepSeek-Coder 6.7B on Bugs4Q dataset
- **Data preparation pipeline:**
 - Train/eval split (70/10/20) of Bugs4Q
 - JSONL-formatted prompt-completion training data
- **Evaluation report** comparing four modes: static, prompt-only, RAG, and fine-tuned
- **Integrated CLI script** (`run_fine_tuned.py`) within the *quantum-bug-rag* framework
- **Structured JSON output:** `bug_likelihood`, `taxonomy_class`, `suspected_location`, `justification`

Chapter 2

Background

2.1 Quantum Bug Benchmarks

Bugs4Q (Zhao et al., 2021) provides real, validated Qiskit bugs with ground-truth labels and serves as the primary evaluation dataset. *QBugs* (Campos & Souto, 2021), hosted on Zenodo, offers a complementary corpus for taxonomy grounding and knowledge-base enrichment.

2.2 LLM-Based Bug Detection

LLM-based bug detection using prompting and RAG works for classical software (Fan et al., 2023) but is unexplored for quantum bugs. LoRA (Hu et al., 2022) and QLoRA (Dettmers et al., 2023) enable parameter-efficient fine-tuning on a single GPU via low-rank adapters and 4-bit quantization.

2.3 Research Gap

No prior work has applied fine-tuning to quantum bug detection. The existing framework provides prompt-only and RAG baselines but lacks a fine-tuning mode. This project introduces the first fine-tuning benchmark for Qiskit bug classification.

Chapter 3

Proposed Technical Approach

3.1 Method Justification

Method choice: Fine-tuning. We choose fine-tuning because quantum bug patterns are highly domain-specific and repetitive knowledge best embedded in model weights rather than retrieved at inference time. Prompting alone lacks specialization, and RAG adds latency while depending on retrieval quality. Fine-tuning internalizes these patterns, enabling faster and more accurate detection on this narrow task.

3.2 Pipeline

1. **Data preparation:** Split Bugs4Q into 70/10/20 train/eval/test sets. Format as prompt-completion JSONL with buggy code as input and structured JSON labels as output.
2. **Model selection:** Fine-tune Llama 3.1 8B and/or DeepSeek-Coder 6.7B using LoRA/QLoRA via HuggingFace Transformers and PEFT.
3. **Training:** LoRA rank 16, learning rate 2e-5, 3–5 epochs with early stopping on validation loss. QLoRA with 4-bit quantization for memory efficiency.
4. **Baseline comparison:** Run prompt-only and RAG baselines (OpenAI API, gpt-4o-mini) for direct comparison.
5. **Output:** Structured JSON: `bug_likelihood`, `taxonomy_class`, `suspected_location`, `justification`.

3.3 Tools and Techniques

- ★ **Fine-tuning:** HuggingFace Transformers, PEFT/LoRA library
- ★ **Base models:** Llama 3.1 8B, DeepSeek-Coder 6.7B
- ★ **Baselines:** OpenAI API (gpt-4o-mini) for prompt-only and RAG
- ★ **Datasets:** Bugs4Q (evaluation), Bugs-QCP (knowledge base)
- ★ **Compute:** Google Colab Pro (A100/T4) or university HPC
- ★ **Framework:** quantum-bug-rag (existing open-source codebase)
- ★ **Language:** Python 3.10+

Chapter 4

Data & Resources

- **Bugs4Q** (public, GitHub, MIT License): ~42 real Qiskit bugs with ground-truth labels. Primary evaluation dataset.
- **Bugs-QCP** (Zenodo 5834281, open access): Taxonomy corpus for RAG knowledge-base enrichment only. Not used for fine-tuning or evaluation.
- **APIs & Libraries:** HuggingFace Transformers + PEFT/LoRA for fine-tuning; OpenAI API (gpt-4o-mini) for baselines.
- **Compute:** Single GPU via Google Colab Pro (A100/T4) or university HPC. QLoRA enables fine-tuning 8B models within 16 GB VRAM.

Chapter 5

Evaluation Plan

All modes (static, prompt-only, RAG, fine-tuned) will be evaluated on the held-out Bugs4Q test split (20%):

- **Accuracy:** Overall taxonomy classification correctness. Target: $\geq 70\%$.
- **F1 Score (macro):** Harmonic mean of precision and recall across all classes. Target: $\geq 60\%$.
- **Precision & Recall:** Per-class breakdown to identify which bug types benefit most.
- **Decision threshold:** Successful if fine-tuned model achieves statistically higher F1 than prompt-only baseline.

Fallback: If dataset size is insufficient, (a) apply data augmentation via code perturbation, and (b) report few-shot prompting with chain-of-thought as a stronger baseline.

Chapter 6

Feasibility & Risks

- **Data preparation:** Bugs4Q train/eval/test split, JSONL formatting, prompt template design.
- **Fine-tuning:** LoRA/QLoRA training on Llama 3.1 8B and DeepSeek-Coder; hyperparameter tuning.
- **Evaluation:** Run all four modes on test split; compute metrics; run baselines.
- **Analysis and report:** Comparative analysis, ablation studies, final write-up.
- **Overfitting:** LoRA reduces trainable parameters to $< 1\%$; early stopping and dropout regularization.
- **Label leakage:** Strict train/eval/test split discipline. Knowledge base from Bugs-QCP only.
- **Compute constraints:** QLoRA 4-bit quantization enables 8B models on a single T4 GPU (16 GB).

Chapter 7

Ethical, Privacy & Security Considerations

- **Bias:** Bugs4Q covers only Qiskit; model may not generalize to Cirq or Q#. We will state this limitation explicitly.
- **PII:** All datasets are open-source with no personally identifiable information.
- **Security:** Quantum bugs in QKD can silently compromise cryptographic guarantees. Automated detection supports quantum software security.
- **Reproducibility:** All code, splits, and configurations shared via the open-source framework.

References

- Campos, J., & Souto, A. (2021). Q Bugs: A collection of reproducible bugs in quantum algorithms and a supporting infrastructure to enable controlled quantum software testing and debugging experiments. *arXiv preprint arXiv:2103.16968*. <https://doi.org/10.48550/arXiv.2103.16968>
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized language models. *Advances in Neural Information Processing Systems*, 36. <https://arxiv.org/abs/2305.14314>
- Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., & Tan, S. H. (2023). Automated repair of programs from large language models. *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. IEEE.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2106.09685>
- Zhao, P., Zhao, J., Miao, Z., & Lan, S. (2021). Bugs4Q: A benchmark of real bugs for quantum programs. *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 1373–1376. IEEE. <https://doi.org/10.1109/ASE51524.2021.9678908>